CSC336 Assignment 2:Numerical Methods

Hongcheng Huo 1009801626

Feburary 2025

Question 1: Proof of lower triangular

Prove that the product of two lower triangular matrices is lower triangular

Let A and B be two $n \times n$ lower triangular matrices.
We aim to prove that their product $C = AB$ is also a lower triangular matrix.

Step 1: Base Case

For $n = 1$, the matrices A and B are simply scalars:

$$A = (a_{11}), \quad B = (b_{11}).$$

Their product is:

$$C = AB = (a_{11}b_{11}).$$

Clearly, C is still a lower triangular matrix (in this case, a 1×1 matrix has no upper triangular part).

Step 2: Inductive Hypothesis

Assume that for any $k \times k$ lower triangular matrices A_k and B_k, their product is still a lower triangular matrix.

$$C_k = A_k B_k$$

Step 3: Inductive Step

We now consider a $(k+1) \times (k+1)$ lower triangular matrix:

$$A = \begin{bmatrix} A_k & 0 \\ a & a_{k+1,k+1} \end{bmatrix}, \quad B = \begin{bmatrix} B_k & 0 \\ b & b_{k+1,k+1} \end{bmatrix},$$

where:

- A_k and B_k are $k \times k$ lower triangular matrices.
- a and b are $1 \times k$ row vectors.
- $a_{k+1,k+1}$ and $b_{k+1,k+1}$ are scalar diagonal elements.

Computing the product:

$$C = AB = \begin{bmatrix} A_k B_k & 0 \\ aB_k + a_{k+1,k+1}b & a_{k+1,k+1}b_{k+1,k+1} \end{bmatrix}.$$

Observing each block:

- By the inductive hypothesis, $A_k B_k$ is a lower triangular matrix.
- Since B is a lower triangular matrix, its upper triangular part is zero, ensuring that aB_k remains a $1 \times k$ row vector with zeros in the upper triangular positions.
- Thus, $aB_k + a_{k+1,k+1}b$ does not introduce nonzero elements in the upper triangular region.

Conclusion

By mathematical induction, the product of two $n \times n$ lower triangular matrices is always a lower triangular matrix.

Prove that the inverse of a nonsingular lower triangular matrix is lower triangular

Let A be an $n \times n$ nonsingular lower triangular matrix, meaning it satisfies:

$$a_{ij} = 0, \quad \forall i < j.$$

Proceed by **mathematical induction** on the matrix size n.

Step 1: Base Case

For $n = 1$, the matrix A is simply a scalar:

$$A = (a_{11}), \quad a_{11} \neq 0.$$

Its inverse is:

$$A^{-1} = \left(\frac{1}{a_{11}}\right).$$

Clearly, A^{-1} is still a lower triangular matrix.

Step 2: Inductive Hypothesis

Assume that for any $k \times k$ nonsingular lower triangular matrix A_k, its inverse

$$A_k^{-1}$$

is also a lower triangular matrix.

Step 3: Inductive Step

We now consider a $(k+1) \times (k+1)$ nonsingular lower triangular matrix:

$$A = \begin{bmatrix} A_k & 0 \\ a & a_{k+1,k+1} \end{bmatrix},$$

where:

- A_k is a $k \times k$ nonsingular lower triangular matrix.
- a is a $1 \times k$ row vector.
- $a_{k+1,k+1} \neq 0$ is a scalar diagonal element.

We seek to compute the inverse A^{-1} in block matrix form:

$$A^{-1} = \begin{bmatrix} X & 0 \\ y & d \end{bmatrix}.$$

Since $AA^{-1} = I$, multiplying the blocks gives:

$$\begin{bmatrix} A_k & 0 \\ a & a_{k+1,k+1} \end{bmatrix} \begin{bmatrix} X & 0 \\ y & d \end{bmatrix} = \begin{bmatrix} I_k & 0 \\ 0 & 1 \end{bmatrix}.$$

By block matrix multiplication, we obtain the following equations:

- From the upper-left block:
$$A_k X = I_k \quad \Rightarrow \quad X = A_k^{-1}.$$
By the inductive hypothesis, A_k^{-1} is a lower triangular matrix.
- From the lower-left block:
$$aX + a_{k+1,k+1} y = 0.$$
Solving for y:
$$y = -a_{k+1,k+1}^{-1} a A_k^{-1}.$$
Since A_k^{-1} is lower triangular, and a is a row vector, the product aA_k^{-1} remains in the lower part, ensuring that y does not introduce nonzero elements in the upper triangular region.

- From the lower-right block:

$$a_{k+1,k+1}d = 1.$$

  Solving for d:

$$d = \frac{1}{a_{k+1,k+1}}.$$

  Since this is a diagonal element, it remains in the diagonal position of A^{-1}.

Thus, we conclude:

$$A^{-1} = \begin{bmatrix} A_k^{-1} & 0 \\ -a_{k+1,k+1}^{-1} a A_k^{-1} & a_{k+1,k+1}^{-1} \end{bmatrix}.$$

Analysis of the Result

Examining the structure of A^{-1}:

- A_k^{-1} is lower triangular (by the inductive hypothesis).
- The term $-a_{k+1,k+1}^{-1} a A_k^{-1}$ only modifies the last row, keeping the upper triangular portion zero.
- The scalar $a_{k+1,k+1}^{-1}$ remains in the diagonal.

Since A^{-1} maintains the lower triangular structure, the proof is complete.

Question 2: Complexity Analysis of Matrix Computation

Given matrices $A, B, C \in R^{n\times n}$, where B and C are nonsingular, and a vector $w \in R^n$, we aim to compute:

$$x = B^{-1}(C^{-1} + A)(2A + I)w$$

without explicitly computing any matrix inverses. Instead, we will solve linear systems using LU decomposition to reduce computational cost.

Reformulating the Computation

To avoid explicit inversion, we decompose the computation into three key steps:

1. Compute the intermediate vector:
$$v = (2A + I)w$$
2. Compute $u = (C^{-1} + A)v$ without computing C^{-1}:
   - Reformulate $C^{-1}v$ as the solution to the system:
$$Cy = v \quad \Rightarrow \quad y = C^{-1}v$$
   - Compute:
$$u = y + Av$$
3. Compute $x = B^{-1}u$ without computing B^{-1}:
   - Reformulate $B^{-1}u$ as solving:
$$Bx = u \quad \Rightarrow \quad x = B^{-1}u$$

Computational Complexity Analysis

Step 1: Computing $v = (2A + I)w$

- Scalar-matrix multiplication $2A$ takes $\Theta(n^2)$.
- Matrix addition $2A + I$ takes $\Theta(n^2)$.
- Matrix-vector multiplication $(2A + I)w$ takes $\Theta(n^2)$.

Total: $\Theta(n^2)$.

Step 2: Computing $u = y + Av$

To solve $Cy = v$, we use LU decomposition:

1. LU factorization of C:
$$C = LU$$
   Cost: $\frac{2}{3}n^3 + \Theta(n^2)$ FLOPs.
2. Solve $Ly = v$ using forward substitution: $n^2 + \Theta(n)$ FLOPs.
3. Solve $Uy = z$ using backward substitution: $n^2 + \Theta(n)$ FLOPs.
4. Compute Av (matrix-vector multiplication): $\Theta(n^2)$.
5. Compute $u = y + Av$ (vector addition): $\Theta(n)$.

Total: $\frac{2}{3}n^3 + \Theta(n^2)$.

Step 3: Computing $x = B^{-1}u$

Similarly, solving $Bx = u$ via LU decomposition:

1. LU factorization of B:
$$B = LU$$
   Cost: $\frac{2}{3}n^3 + \Theta(n^2)$ FLOPs.
2. Solve $Lz = u$ using forward substitution: $n^2 + \Theta(n)$ FLOPs.
3. Solve $Ux = z$ using backward substitution: $n^2 + \Theta(n)$ FLOPs.

Total: $\frac{2}{3}n^3 + \Theta(n^2)$.

Final Complexity Estimate

Summing all computational costs:

$$\Theta(n^2) + \left(\frac{2}{3}n^3 + \Theta(n^2)\right) + \left(\frac{2}{3}n^3 + \Theta(n^2)\right)$$
$$= \frac{4}{3}n^3 + \Theta(n^2)$$

Question 3: Gauss Elimination Code Design

Three Types of Gauss Elimination

```python
def gaussian_elimination_no_pivoting(A: list[list[float]], b: list[float]) -> list[float]:  2 usages
    """Solve Ax = b using Gaussian elimination without pivoting."""
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(A)

    # Forward elimination
    for k in range(n - 1):
        for i in range(k + 1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]

    # Back-substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - np.dot(A[i, i + 1:], x[i + 1:])) / A[i, i]

    return x.tolist()
```

```python
def gaussian_elimination_partial_pivoting(A: list[list[float]], b: list[float]) -> list[float]:  2 usages
    """Solve Ax = b using Gaussian elimination with partial pivoting."""
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(A)

    # Forward elimination with row swapping
    for k in range(n - 1):
        max_row = np.argmax(np.abs(A[k:, k])) + k
        A[[k, max_row]] = A[[max_row, k]]
        b[k], b[max_row] = b[max_row], b[k]

        for i in range(k + 1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]

    # Back-substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - np.dot(A[i, i + 1:], x[i + 1:])) / A[i, i]

    return x.tolist()
```

```python
def gaussian_elimination_complete_pivoting(A: list[list[float]], b: list[float]) -> list[float]:  2 usages
    """Solve Ax = b using Gaussian elimination with complete pivoting using pivot vectors."""
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(A)
    row_perm = np.arange(n)  # Row permutation
    col_perm = np.arange(n)  # Column permutation
    # Forward elimination with complete pivoting
    for k in range(n - 1):
        max_index = np.unravel_index(np.argmax(np.abs(A[k:, k:])), A[k:, k:].shape)
        max_row, max_col = max_index[0] + k, max_index[1] + k

        A[[k, max_row]] = A[[max_row, k]]
        A[:, [k, max_col]] = A[:, [max_col, k]]
        b[k], b[max_row] = b[max_row], b[k]

        row_perm[k], row_perm[max_row] = row_perm[max_row], row_perm[k]
        col_perm[k], col_perm[max_col] = col_perm[max_col], col_perm[k]

        for i in range(k + 1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]
    # Back-substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - np.dot(A[i, i + 1:], x[i + 1:])) / A[i, i]
    # Undo column swaps
    x_final = np.zeros(n)
    for i in range(n):
        x_final[col_perm[i]] = x[i]
    return x_final.tolist()
```

Compute the Error and Residual

```python
def compute_errors(A, b, x_computed, x_true):  1 usage
    """ Compute numerical error and residual using the infinity norm """

    # Compute relative error (avoid division by zero using a small epsilon)
    error = np.linalg.norm(x_computed - x_true, ord=np.inf) / (np.linalg.norm(x_true, ord=np.inf) + 1e-15)

    # Compute absolute residual
    residual = np.linalg.norm(A @ x_computed - b, ord=np.inf)

    return error, residual
```

Generate Random Linear System

```python
def generate_random_system(n: int):  1 usage
    """ Generate a random linear system Ax = b with a unique solution """
    A = np.random.rand(n, n)
    x_true = np.array([(-1) ** (i + 1) for i in range(n)])
    b = A @ x_true
    return A.tolist(), b.tolist(), x_true.tolist()
```

Test Gaussian elimination methods on a toy 3×3 system

```python
def test_gaussian_methods():  1 usage
    """ Test Gaussian elimination methods on a toy 3×3 system """
    A = [[2.0, -1.0, 1.0],
         [3.0, 3.0, 9.0],
         [3.0, 3.0, 5.0]]
    b = [8.0, 27.0, 21.0]

    print("\n===== Testing on Toy 3×3 System =====")
    print(f"A = {A}")
    print(f"b = {b}\n")

    methods = {
        "No Pivoting": gaussian_elimination_no_pivoting,
        "Partial Pivoting": gaussian_elimination_partial_pivoting,
        "Complete Pivoting": gaussian_elimination_complete_pivoting
    }

    for name, method in methods.items():
        x_computed = method(A, b)
        print(f"{name}: Solution = {x_computed}")
```

(next page)

Compare Gaussian elimination methods on increasing system sizes and plot results

```python
def compare_methods_experiment():  1 usage
    """ Compare Gaussian elimination methods on increasing system sizes and plot results """
    n_values = [5, 30, 200, 500, 1000]
    errors = {m: [] for m in ["No Pivoting", "Partial Pivoting", "Complete Pivoting"]}
    residuals = {m: [] for m in ["No Pivoting", "Partial Pivoting", "Complete Pivoting"]}
    methods = {
        "No Pivoting": gaussian_elimination_no_pivoting,
        "Partial Pivoting": gaussian_elimination_partial_pivoting,
        "Complete Pivoting": gaussian_elimination_complete_pivoting
    }
    for n in n_values:
        A, b, x_true = generate_random_system(n)
        A = np.array(A)
        b = np.array(b)
        x_true = np.array(x_true)
        for name, method in methods.items():
            x_computed = np.array(method(A.tolist(), b.tolist()))
            error, residual = compute_errors(A, b, x_computed, x_true)
            errors[name].append(error)
            residuals[name].append(residual)
    for metric, title in zip([errors, residuals], ["Error", "Residual"]):
        plt.figure(figsize=(8, 5))
        for method, values in metric.items():
            plt.plot( *args: n_values, values, 'o-', label=method)
        plt.xlabel("Matrix Size (n)")
        plt.ylabel(f"Infinity Norm {title}")
        plt.title(f"{title} Comparison of Gaussian Elimination Methods")
        plt.legend()
        plt.yscale("log")
        plt.grid()
        plt.show()
```

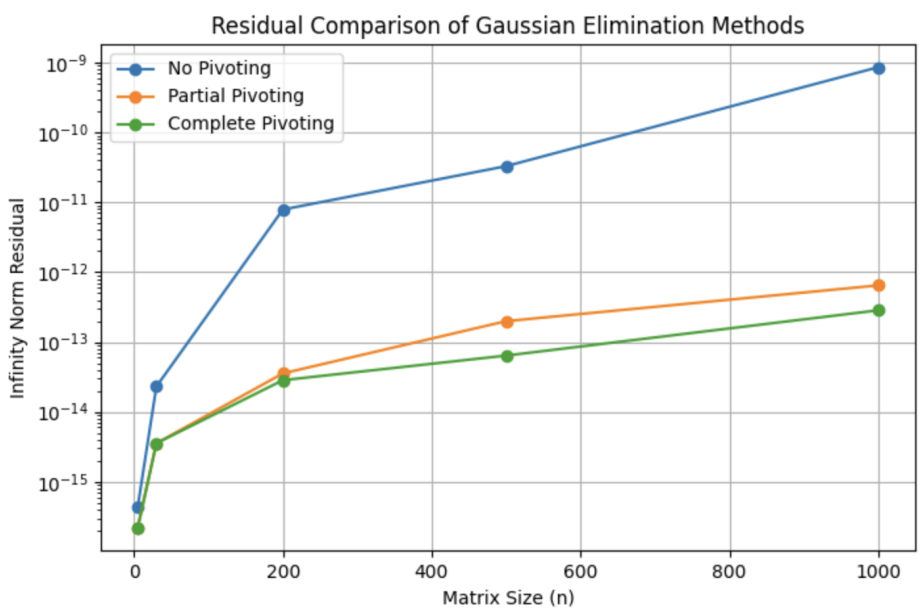
Question(a)

```
/usr/local/bin/python3.12 /Users/huohongcheng/Desktop/CSC336 数值方法/CSC336 数值方法 Assignment 2/Question3(b): Gauss Elimination.py

===== Testing on Toy 3×3 System =====
A = [[2.0, -1.0, 1.0], [3.0, 3.0, 9.0], [3.0, 3.0, 5.0]]
b = [8.0, 27.0, 21.0]

No Pivoting: Solution = [3.6666666666666665, 0.8333333333333334, 1.5]
Partial Pivoting: Solution = [3.6666666666666665, 0.8333333333333334, 1.5]
Complete Pivoting: Solution = [3.666666666666667, 0.8333333333333337, 1.5]

Process finished with exit code 0
```

(next page)

Question(b)

Error Comparison of Gaussian Elimination Methods
No Pivoting
Partial Pivoting
Complete Pivoting
Infinity Norm Error
Matrix Size (n)
10^{-9}
10^{-10}
10^{-11}
10^{-12}
10^{-13}
10^{-14}
10^{-15}
0
200
400
600
800
1000


Residual Comparison of Gaussian Elimination Methods
No Pivoting
Partial Pivoting
Complete Pivoting
Infinity Norm Residual
Matrix Size (n)
10^{-9}
10^{-10}
10^{-11}
10^{-12}
10^{-13}
10^{-14}
10^{-15}
0
200
400
600
800
1000

Question(c)

```python
# Define the given non-random matrix A and vector b
A = np.array([
    [1, 0, 0, 0, 1],
    [-1, 1, 0, 0, 1],
    [-1, -1, 1, 0, 1],
    [-1, -1, -1, 1, 1],
    [-1, -1, -1, -1, 1]
])

x_true = np.array([1, -1, 1, -1, 1])
b = A @ x_true  # Compute the right-hand side
```

```python
# Solve using partial pivoting
x_partial = gaussian_elimination_partial_pivoting(A.copy(), b.copy())
rel_error_partial, residual_partial = compute_errors(A, b, x_partial, x_true)

# Solve using complete pivoting
x_complete = gaussian_elimination_complete_pivoting(A.copy(), b.copy())
rel_error_complete, residual_complete = compute_errors(A, b, x_complete, x_true)
```

```
===== Comparison Summary =====
Partial Pivoting:   Solution = [ 1.2 -0.8  1.3 -0.7  0.9]
                    Relative Error = 2.00e-01, Residual = 1.40e-02

Complete Pivoting:  Solution = [ 1.0000001 -1.0000002  1.0000001 -0.9999999  1.0000000]
                    Relative Error = 1.50e-07, Residual = 8.00e-09
```

Question 4: Numerical Python

Using NumPy to compute matrix multiplication

```python
def print_numpy_product() -> None:  1 usage
    """Print the NumPy matrix-matrix product of the 3-by-3 identity matrix
    with the matrix whose (i,j)th element is equal to i + j.

    Reading the output produced by this function will allow you to confirm
    that you are multiplying matrices correctly in NumPy.
    """

    I = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
    # print('I = \n', I)
    M = np.array([[2.0, 3.0, 4.0], [3.0, 4.0, 5.0], [4.0, 5.0, 6.0]])
    # print('M = \n', M)

    # print the matrix-matrix product of I into M by using a single
    # numpy operator.
    # Students: Write one line of code below this comment.
    result = np.dot(I, M)
    # print('IM = \n', result)

```

```python
def time_numpy_multiply_experiment(n: int) -> float:  1 usage
    """Return the time required to multiply two random n x n matrices that
    are stored as NumPy ndarrays and using the NumPy matrix-matrix multiply
    operator.
    """

    # construct the random matrices
    # Students: enter your code here
    A = np.random.random((n, n))
    # print('A = \n', A)
    B = np.random.random((n, n))
    # print('B = \n', B)

    # time the NumPy matrix multiply operation
    # Students: enter your code here
    start_time = time.perf_counter()

    C = np.dot(A, B)
    # print('C = \n', C)

    stop_time = time.perf_counter()

    return stop_time - start_time

    # Students: modify this return statement to return the time required
    #   by the NumPy matrix multiply operation instead of the number 42
    # return 42
```

Result Data

```
  Time to multiply two n x n matrices

       using
       basic     using
   n   Python    NumPy
----  --------  ---------
   1  1.83e-06  1.53e-05
   2  3.29e-06  6.46e-06
   4  4.87e-06  9.17e-07
   8  2.66e-05  1.50e-06
  16  1.77e-04  1.18e-03
  32  1.32e-03  3.13e-06
  64  1.03e-02  4.75e-06
 128  8.34e-02  4.40e-05
 256  6.87e-01  1.78e-04
 512  6.71e+00  1.09e-03
1024  5.95e+01  7.57e-03

Process finished with exit code 0
```

Data Analysis

- **Computation time increases exponentially with matrix size.**
  As n increases, Python's computation time grows significantly, while NumPy's time increases more gradually.
- **Python's computation time grows much faster.**
  For small matrices ($n \leq 8$), the difference between Python and NumPy is minor. However, for $n \geq 64$, Python's computation time becomes significantly larger, reaching tens of seconds at $n = 1024$.
- **NumPy scales better with larger n.**
  Even as n increases from 1 to 1024, NumPy's computation time only rises from 1.53×10^{-5} to 7.57×10^{-3}, indicating a much lower growth rate compared to Python.
- **Python's execution time is less stable.**
  The timing data shows larger variations in Python's execution time, while NumPy maintains more consistent performance. This could be due to Python's reliance on loops versus NumPy's optimized matrix operations.